

```
import pandas as pd
import numpy as np
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import os

folder_path = "/content/drive/MyDrive/Cycler Data/2017-05-12"
csv_files = [file for file in os.listdir(folder_path) if file.endswith('.csv')]

print(csv_files)
#df = pd.read_csv('/content/drive/MyDrive/Cycler Data/2017-05-12_6C-50per_3_6C-CH36.csv')
```

```
['2017-05-12_6C-50per_3_6C-CH36.csv', '2017-05-12_6C-40per_3_6C-CH34.csv', '2017-05-12_7C-40per_3C-CH37.csv', '2017-05-12_5_4C-8
```

```
len(csv_files)
```

33

```
use_cols = [
    "Cycle_Index",
    "Voltage",
    "Current",
    "Charge_Capacity",
    "Discharge_Capacity",
    "Temperature",
    "Internal_Resistance"
]
```

```
csv_paths = [os.path.join(folder_path, file_name) for file_name in csv_files]
```

```
df_list = [pd.read_csv(path, usecols=lambda col: col in use_cols) for path in csv_paths]
#df_list.append(df) # add new DataFrame
#df_list.extend([df1, df2]) # add multiple
#df_list.insert(0, df) # insert at position
#df_list.pop() # remove last
#df_list.remove(df) # remove specific df
#df_list.clear() # empty list
```

```
df_list[0].head()
```

	Cycle_Index	Current	Voltage	Charge_Capacity	Discharge_Capacity	Internal_Resistance	Temperature
0	0.0	0.0	3.301788	0.0	0.0	0.022012	30.457119
1	0.0	0.0	3.301788	0.0	0.0	0.022012	30.457119
2	0.0	0.0	3.301751	0.0	0.0	0.022012	30.445261
3	0.0	0.0	3.301784	0.0	0.0	0.022012	30.500921
4	0.0	0.0	3.301804	0.0	0.0	0.022012	30.500921

```
print(f"Discharge Capacity Range: {df_list[0]['Discharge_Capacity'].min()} - {df_list[0]['Discharge_Capacity'].max()}")
print(f"Charge Capacity Range: {df_list[0]['Charge_Capacity'].min()} - {df_list[0]['Charge_Capacity'].max()}")
```

```
Discharge Capacity Range: 0.0 - 1.6988505
Charge Capacity Range: 0.0 - 1.1737735
```

```
df_list[0].info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 919051 entries, 0 to 919050
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Cycle_Index      919051 non-null float64
1   Current          919051 non-null float64
```

```

2 Voltage          919051 non-null float64
3 Charge_Capacity  919051 non-null float64
4 Discharge_Capacity 919051 non-null float64
5 Internal_Resistance 919051 non-null float64
6 Temperature      919051 non-null float64
dtypes: float64(7)
memory usage: 49.1 MB

```

```
df_list[0].shape
```

```
(919051, 7)
```

```
df_list[0].describe()
```

	Cycle_Index	Current	Voltage	Charge_Capacity	Discharge_Capacity	Internal_Resistance	Temperature
count	919051.000000	919051.000000	919051.000000	919051.000000	9.190510e+05	919051.000000	919051.000000
mean	432.446211	0.094878	3.106731	0.839600	2.810022e-01	0.017369	30.971779
std	252.551549	3.738575	0.533847	0.360521	4.114450e-01	0.000645	0.940417
min	0.000000	-4.402013	1.995598	0.000000	0.000000e+00	0.000000	25.430899
25%	214.000000	-4.399846	2.868634	0.809195	1.305898e-09	0.016960	30.359972
50%	431.000000	0.057050	3.351889	1.017111	4.466320e-09	0.017120	30.624111
75%	650.000000	2.043119	3.525281	1.078298	6.244384e-01	0.017661	31.384649
max	876.000000	6.607799	3.611227	1.173774	1.698851e+00	0.022012	34.404381

```

df_list[0].isnull().sum()
#df = df.dropna(subset=['Voltage','Current']) for remove rows with missing voltage/current

```

	0
Cycle_Index	0
Current	0
Voltage	0
Charge_Capacity	0
Discharge_Capacity	0
Internal_Resistance	0
Temperature	0

```
dtype: int64
```

```

# voltage never becomes zero it only range from lowest to highest.
index = []
for i, df_item in enumerate(df_list):
    if 'Voltage' in df_item.columns:
        # Check if there are any rows where 'Voltage' is less than or equal to 0
        if (df_item['Voltage'] <= 0).any():
            index.append(i)

display(index)

```

```
[]
```

```
#removing Battery startup dynamics, initial behavior analyze, making all datapoint valid.
```

```

n = len(df_list)

for i in range(n):
    df = df_list[i]
    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100) & (df['Cycle_Index'] > 0)]
    df_list[i] = df

```

✓ for adding processed data to drive

@title Default title text

```
output_folder = "/content/drive/MyDrive/CyclerData_2017-05-12_processed"
os.makedirs(output_folder, exist_ok=True)
import pandas as pd
```

```
use_cols = ["Cycle_Index", "Voltage", "Current", "Charge_Capacity", "Discharge_Capacity", "Temperature", "Internal_Resistance" ]
```

```
for path in csv_paths:
    try:
        # Read only needed columns
        df = pd.read_csv(path, usecols=lambda col: col in use_cols)
```

```
        # Reduce memory
        df = df.astype("float32")

        # Get original filename
        filename = os.path.basename(path)

        # Save cleaned file
        output_path = os.path.join(output_folder, filename)
        df.to_csv(output_path, index=False)

        print(f"Processed: {filename}")

    except Exception as e:
        print(f"Error in {path}: {e}")
```

```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.widgets import Slider

# 📁 store processed data
cycle_counts_list = []
battery_lives = []

for df in df_list:
    cycle_counts = df['Cycle_Index'].value_counts().sort_index()
    life = df['Cycle_Index'].max()

    cycle_counts_list.append(cycle_counts)
    battery_lives.append(life)

battery_lives = np.array(battery_lives)

# 🎨 categorical colors (distinct for each battery)
cmap = plt.cm.get_cmap('tab20', len(cycle_counts_list))

# 🖼 figure
fig, ax = plt.subplots(figsize=(20,6))
plt.subplots_adjust(bottom=0.2)

lines = []

# plot all batteries
for i, cycle_counts in enumerate(cycle_counts_list):

    color = cmap(i) # 🏷 different color for each battery

    line, = ax.plot(cycle_counts.index,
                    cycle_counts.values,
                    color=color,
                    alpha=0.7,
                    label=f'Battery {i}')

    lines.append(line)

# axis labels
ax.set_xlabel("Cycle Index")
ax.set_ylabel("Number of Data Points")
ax.set_title("Data Points vs Cycle Index (All Batteries)")
```

```

ax.grid(True, linestyle='--', alpha=0.5)

# legend (optional but useful now)
ax.legend(loc='upper right', fontsize=8)

# 🛠 slider axis
ax_slider = plt.axes([0.2, 0.05, 0.6, 0.03])
slider = Slider(ax_slider,
                'Battery Index',
                0,
                len(cycle_counts_list)-1,
                valinit=0,
                valstep=1)

# 🔄 update function
def update(val):

    idx = int(slider.val)

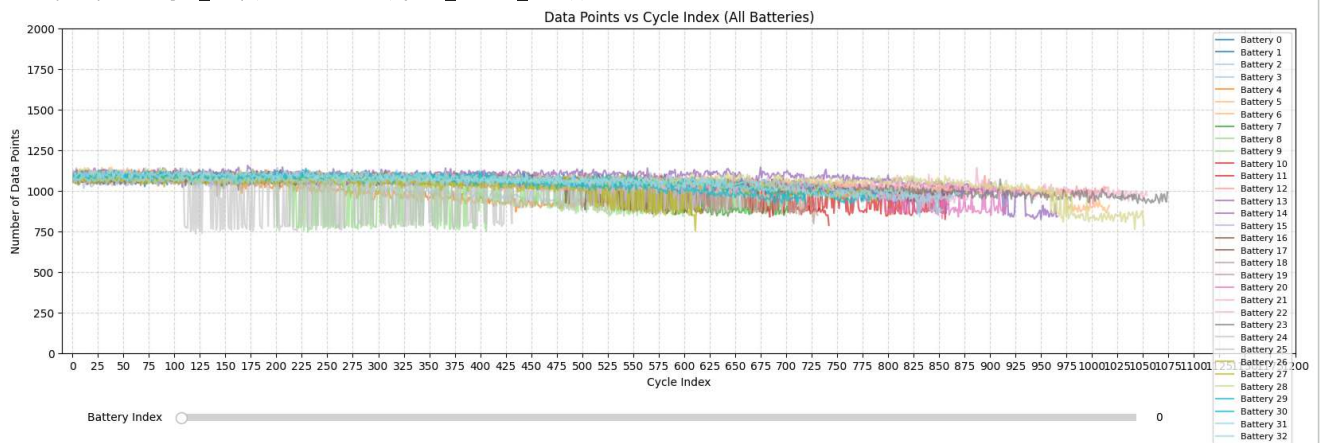
    for i, line in enumerate(lines):
        if i == idx:
            line.set_alpha(1)
            line.set_linewidth(3)
        else:
            line.set_alpha(0.1)
            line.set_linewidth(1)

    fig.canvas.draw_idle()

slider.on_changed(update)
ax.set_xlim(-10, 1200)
ax.set_ylim(0, 2000)
ax.set_xticks(np.arange(0, 1200 + 25, 25))
plt.show()

```

/tmp/ipykernel_3001/2299709800.py:19: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and w
cmap = plt.cm.get_cmap('tab20', len(cycle_counts_list))



```

#for faulty data in accordance to above graph like happend to 29 and we removed it so this is a correct one.
df = df_list[29]
# 📊 count data points per cycle
cycle_counts = df['Cycle_Index'].value_counts().sort_index()

# 📈 plot
plt.figure(figsize=(20,6))

```

```
plt.plot(cycle_counts.index,
         cycle_counts.values,
         color='blue',
         linewidth=2)

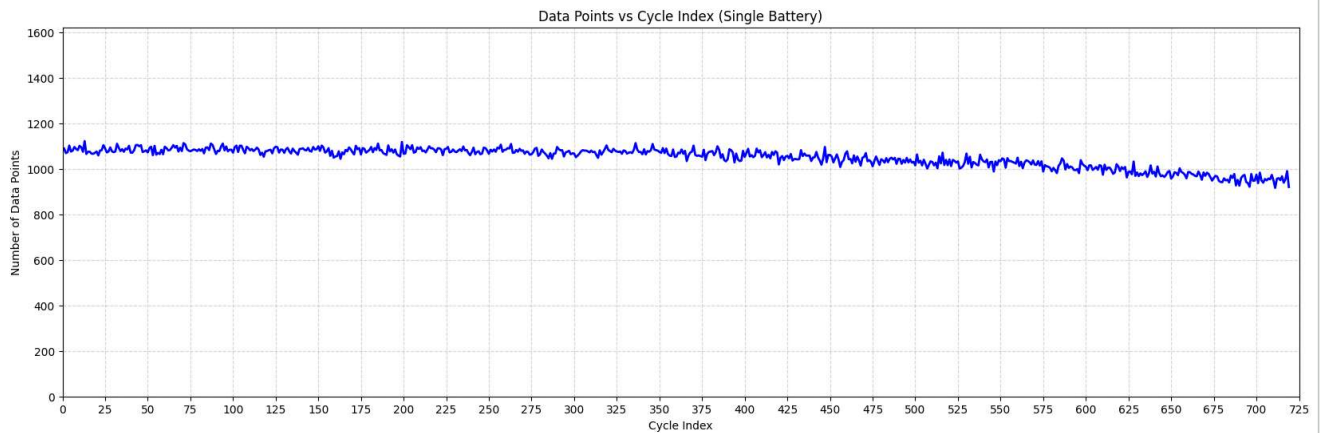
# labels
plt.xlabel("Cycle Index")
plt.ylabel("Number of Data Points")
plt.title("Data Points vs Cycle Index (Single Battery)")

# grid
plt.grid(True, linestyle='--', alpha=0.5)

# optional limits (adjust if needed)
plt.xlim(0, df['Cycle_Index'].max())
plt.ylim(0, cycle_counts.max() + 500)

# optional ticks
plt.xticks(np.arange(0, df['Cycle_Index'].max()+25, 25))

plt.show()
```



```
#max discharge capacity for each cycle index
import numpy as np
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

# high contrast colormap
colors = plt.cm.viridis(np.linspace(0, 1, len(df_list)))

for i, df in enumerate(df_list):

    # cleaning
    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100)]

    # ML usable data
    df_ml = df[df['Cycle_Index'] > 0]

    if df_ml.empty:
        continue

    # capacity fade
    cycle_capacity = df_ml.groupby('Cycle_Index')['Discharge_Capacity'].max()

    # plot
    plt.plot(cycle_capacity.index, cycle_capacity.values,
```

```

        color=colors[i],
        linewidth=2,
        alpha=0.9,
    )

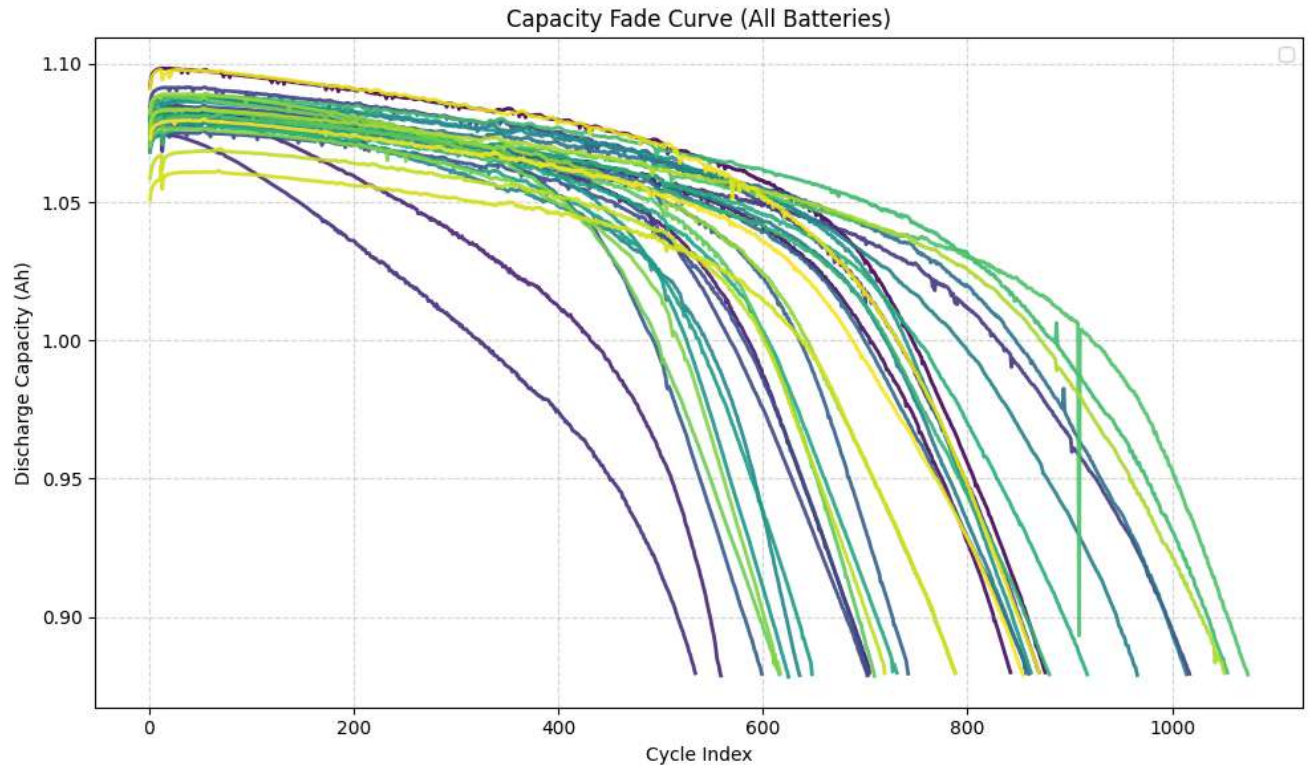
# styling
plt.title("Capacity Fade Curve (All Batteries)")
plt.xlabel("Cycle Index")
plt.ylabel("Discharge Capacity (Ah)")

plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.tight_layout()

plt.show()

```

/tmp/ipykernel_3001/3859847507.py:37: UserWarning: No artists with labels found to put in legend. Note that artists whose label
plt.legend()



```

import numpy as np
import matplotlib.pyplot as plt

ratios = []

for df in df_list:

    # cleaning
    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100)]
    df_ml = df[df['Cycle_Index'] > 0]

    if df_ml.empty:
        continue

    # cycle 2 capacity
    c2 = df_ml[df_ml['Cycle_Index'] == 2]
    # cycle 100 capacity
    c100 = df_ml[df_ml['Cycle_Index'] == 100]

    if c2.empty or c100.empty:
        continue

    cap2 = c2['Discharge_Capacity'].max()
    cap100 = c100['Discharge_Capacity'].max()

    # ratio

```

```

ratio = cap100 / cap2

ratios.append(ratio)

ratios = np.array(ratios)

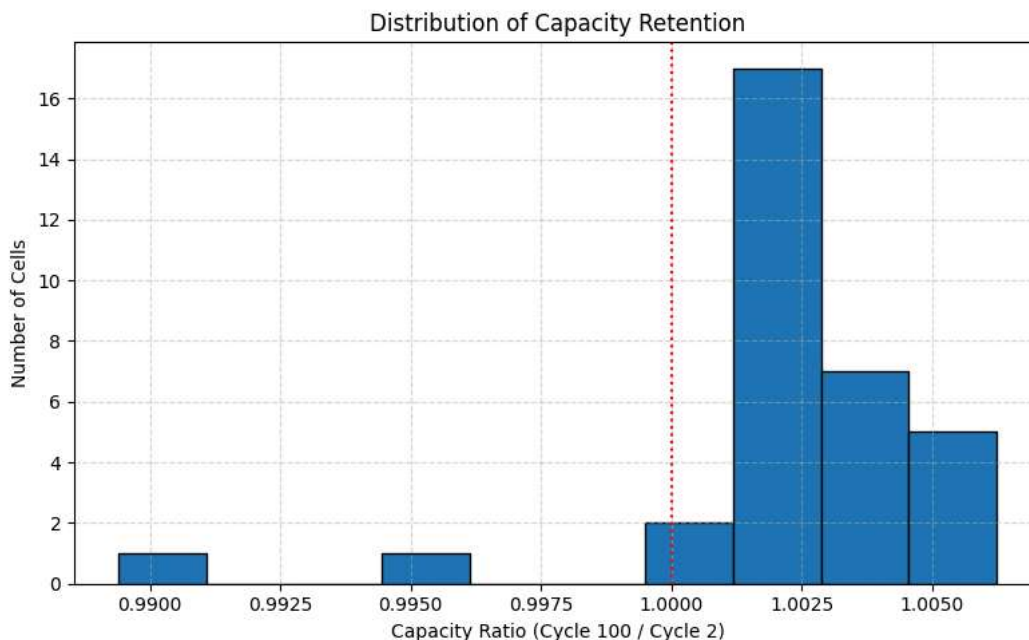
# 🎨 histogram
plt.figure(figsize=(8,5))

plt.hist(ratios, bins=10, edgecolor='black')

plt.xlabel("Capacity Ratio (Cycle 100 / Cycle 2)")
plt.ylabel("Number of Cells")
plt.title("Distribution of Capacity Retention")

plt.grid(True, linestyle='--', alpha=0.5)
plt.axvline(x=1, color='r', linestyle='dotted')
plt.tight_layout()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

colors = plt.cm.plasma(np.linspace(0, 1, len(df_list)))

caps = []
lives = []

for i, df in enumerate(df_list):

    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100)]
    df_ml = df[df['Cycle_Index'] > 0]

    if df_ml.empty:
        continue

    cycle_data = df_ml[df_ml['Cycle_Index'] == 100]
    if cycle_data.empty:
        continue

    cap = cycle_data['Discharge_Capacity'].max()
    life = df_ml['Cycle_Index'].max()

    caps.append(cap)
    lives.append(life)

plt.scatter(cap, life,
            color=colors[i],

```

```

s=80,
alpha=0.9)

# 🌟 correlation
caps = np.array(caps)
lives = np.array(lives)

corr = np.corrcoef(caps, np.log(lives))[0,1]

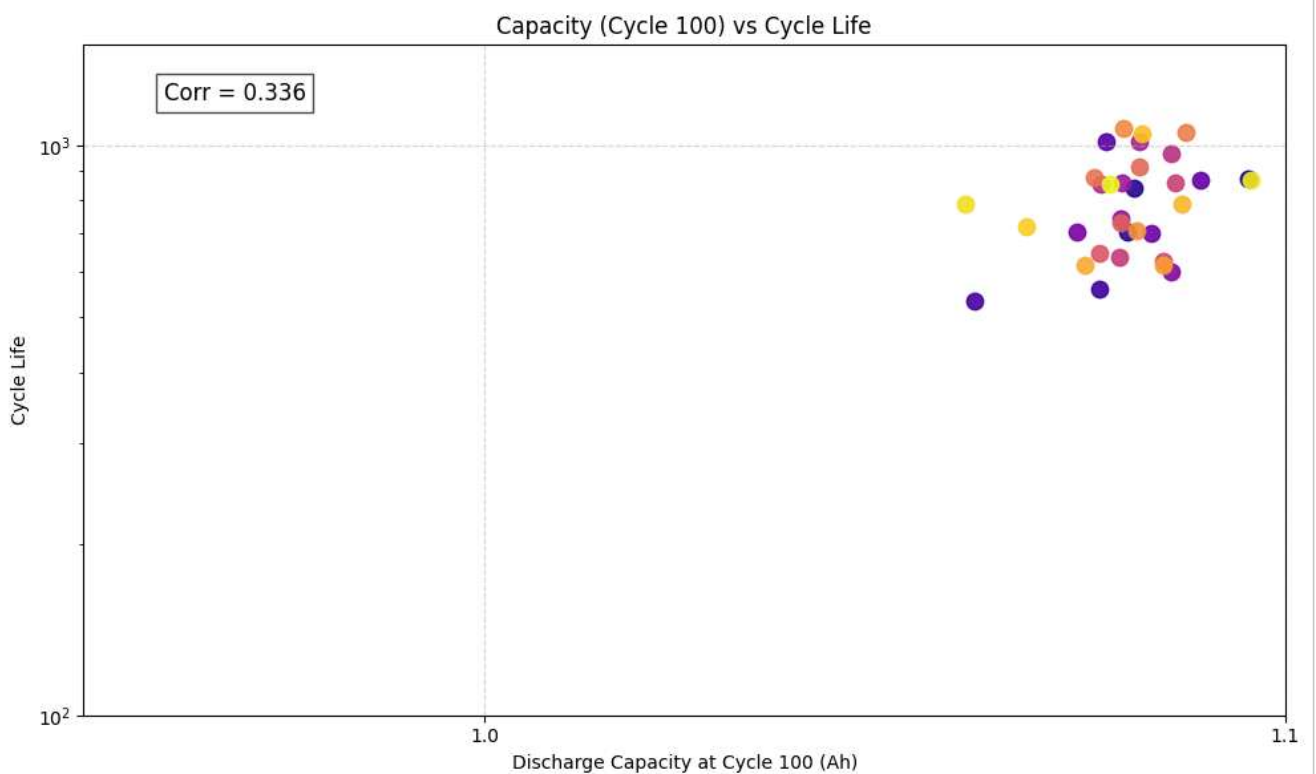
# ✨ text on graph
plt.text(0.96, 1200, f'Corr = {corr:.3f}', fontsize=12,
        bbox=dict(facecolor='white', alpha=0.7))

# styling
plt.yscale('log')
plt.xlim(0.95,1.1)
plt.ylim(10**2, 1500)
plt.xticks([1,1.1])
plt.xlabel("Discharge Capacity at Cycle 100 (Ah)")
plt.ylabel("Cycle Life")
plt.title("Capacity (Cycle 100) vs Cycle Life")

plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()

plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

colors = plt.cm.plasma(np.linspace(0, 1, len(df_list)))

caps = []
lives = []

for i, df in enumerate(df_list):

    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100)]
    df_m1 = df[df['Cycle_Index'] > 0]

    if df_m1.empty:
        continue

```



```

cycle_data = df_ml[df_ml['Cycle_Index'] == 2]
if cycle_data.empty:
    continue

cap = cycle_data['Discharge_Capacity'].max()
life = df_ml['Cycle_Index'].max()

caps.append(cap)
lives.append(life)

plt.scatter(cap, life,
            color=colors[i],
            s=80,
            alpha=0.9)

# 🔥 correlation
caps = np.array(caps)
lives = np.array(lives)

corr = np.corrcoef(caps, np.log(lives))[0,1]

# 🌟 text on graph
plt.text(0.96, 1200, f'Corr = {corr:.3f}', fontsize=12,
        bbox=dict(facecolor='white', alpha=0.7))

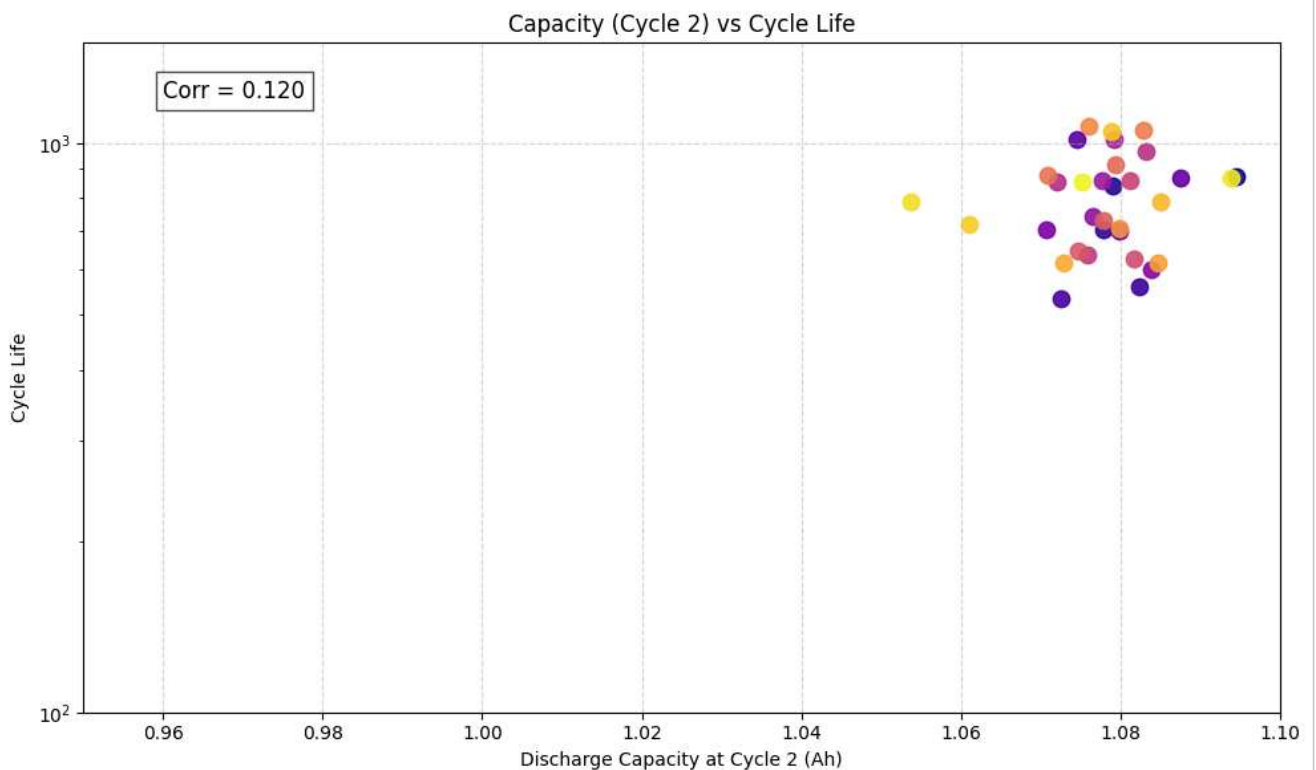
# styling
plt.yscale('log')
plt.xlim(0.95,1.1)
plt.ylim(10**2, 1500)

plt.xlabel("Discharge Capacity at Cycle 2 (Ah)")
plt.ylabel("Cycle Life")
plt.title("Capacity (Cycle 2) vs Cycle Life")

plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()

plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import linregress

```

```

plt.figure(figsize=(10,6))

colors = plt.cm.plasma(np.linspace(0, 1, len(df_list)))

slopes_all = []
life_all = []

for i, df in enumerate(df_list):

    # cleaning
    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100)]
    df_ml = df[df['Cycle_Index'] > 0]

    if df_ml.empty:
        continue

    # capacity curve
    cycle_capacity = df_ml.groupby('Cycle_Index')['Discharge_Capacity'].max()

    # check cycles exist
    if not all(c in cycle_capacity.index for c in range(95, 101)):
        continue

    # slope calculation
    x = np.array(range(95, 101))
    y = cycle_capacity.loc[95:100].values

    slope, _, _, _ = linregress(x, y)

    # cycle life (80% threshold)
    try:
        cycle_life = cycle_capacity[cycle_capacity < 0.8].index[0]
    except:
        cycle_life = cycle_capacity.index.max()

    slopes_all.append(slope)
    life_all.append(cycle_life)

    # scatter plot
    plt.scatter(slope, cycle_life,
                color=colors[i],
                s=80,
                alpha=0.9)

log_cycle_lives = np.log(life_all)

correlation = np.corrcoef(slopes_all, log_cycle_lives)[0,1]

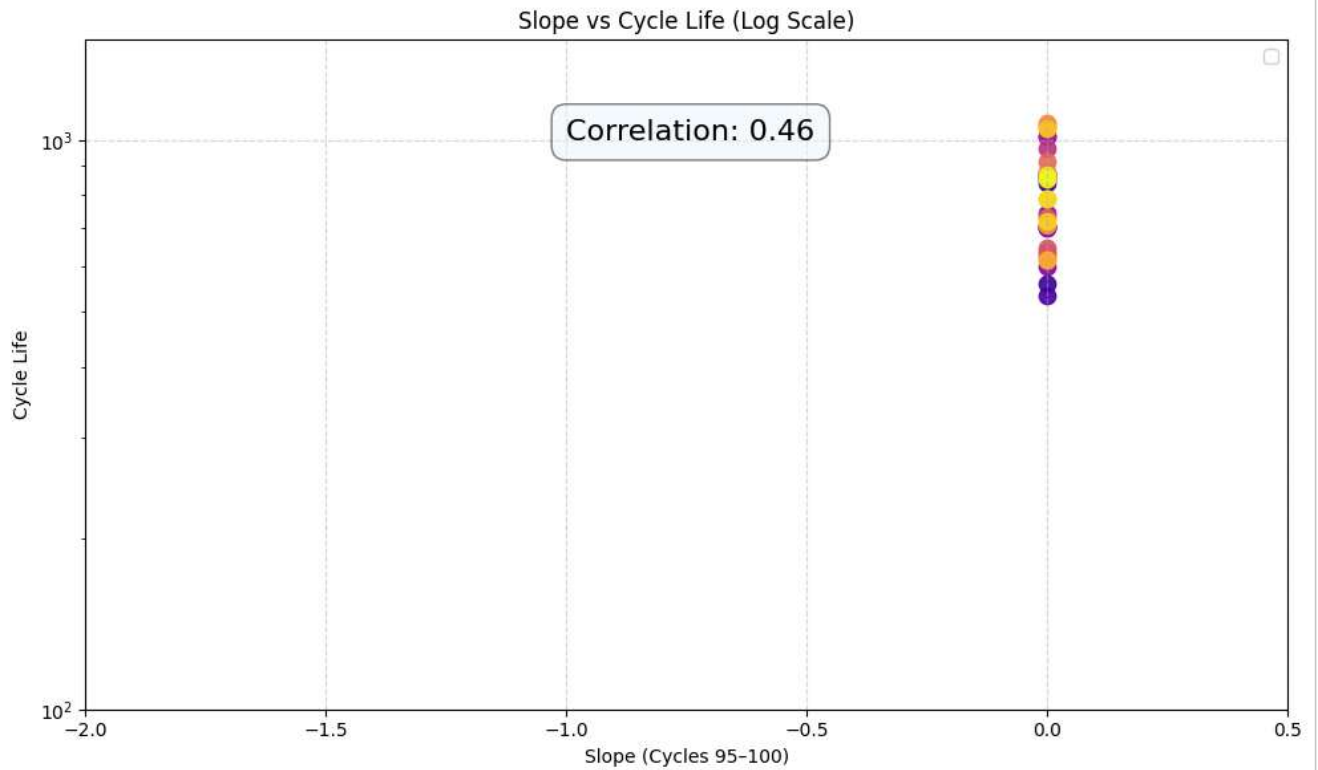
# styling (same clean look)
plt.yscale('log')
plt.xlim(-2,0.5)
plt.ylim(10**2,1500)
plt.xlabel("Slope (Cycles 95-100)")
plt.ylabel("Cycle Life")
plt.title("Slope vs Cycle Life (Log Scale)")
plt.text(-1, 10**3, "Correlation: {:.2f}".format(correlation), fontsize=16, bbox=dict(boxstyle="round,pad=0.5", fc="aliceblue",

plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.tight_layout()

plt.show()

```

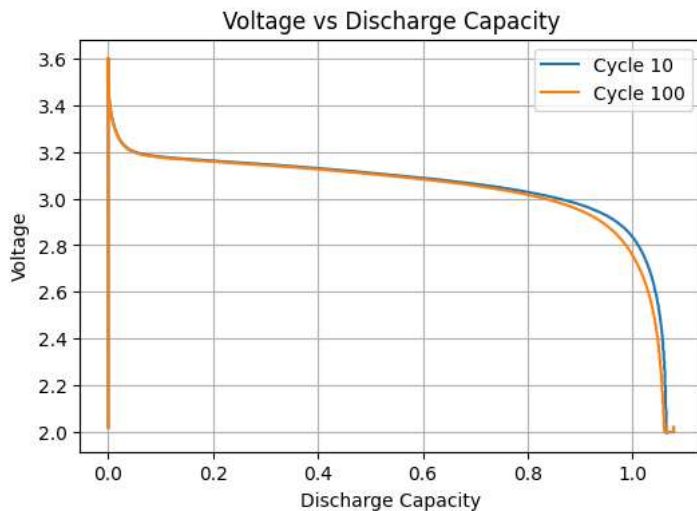
/tmp/ipykernel_3001/4030763844.py:65: UserWarning: No artists with labels found to put in legend. Note that artists whose label plt.legend()



#Voltage curve shift hota hai jab battery degrade hoti hai.
 #10th vs 100th cycle voltage vs capacity degradation curve

```
cycle10 = df_ml[df_ml['Cycle_Index']==10]
cycle100 = df_ml[df_ml['Cycle_Index']==100]

plt.figure(figsize=(6,4))
plt.plot(cycle10['Discharge_Capacity'], cycle10['Voltage'], label='Cycle 10')
plt.plot(cycle100['Discharge_Capacity'], cycle100['Voltage'], label = 'Cycle 100')
plt.xlabel('Discharge Capacity')
plt.ylabel('Voltage')
plt.title('Voltage vs Discharge Capacity')
plt.legend()
plt.grid(True)
plt.show()
```



```
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

# voltage grid same for all
voltage_grid = np.linspace(2.0, 3.5, 500)

plt.figure(figsize=(6,8))

# high contrast color map
colors = plt.cm.plasma(np.linspace(0, 1, len(df_list)))

for i, df in enumerate(df_list):

    # cleaning (important)
    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100)]

    # select cycles
    df1 = df[(df['Cycle_Index']==10) & (df['Current'] < 0)]
    df2 = df[(df['Cycle_Index']==100) & (df['Current'] < 0)]

    # skip if data missing
    if df1.empty or df2.empty:
        continue

    # sorting + duplicate removal
    df1 = df1.sort_values('Voltage').drop_duplicates('Voltage')
    df2 = df2.sort_values('Voltage').drop_duplicates('Voltage')

    try:
        # interpolation
        f1 = interp1d(df1['Voltage'], df1['Discharge_Capacity'],
                      fill_value="extrapolate")
        f2 = interp1d(df2['Voltage'], df2['Discharge_Capacity'],
                      fill_value="extrapolate")

        q1 = f1(voltage_grid)
        q2 = f2(voltage_grid)

        delta_q = q2 - q1

        # plot with contrast
        plt.plot(delta_q, voltage_grid,
                 color=colors[i],
                 alpha=0.9,
                 linewidth=2,)

    except:
        continue

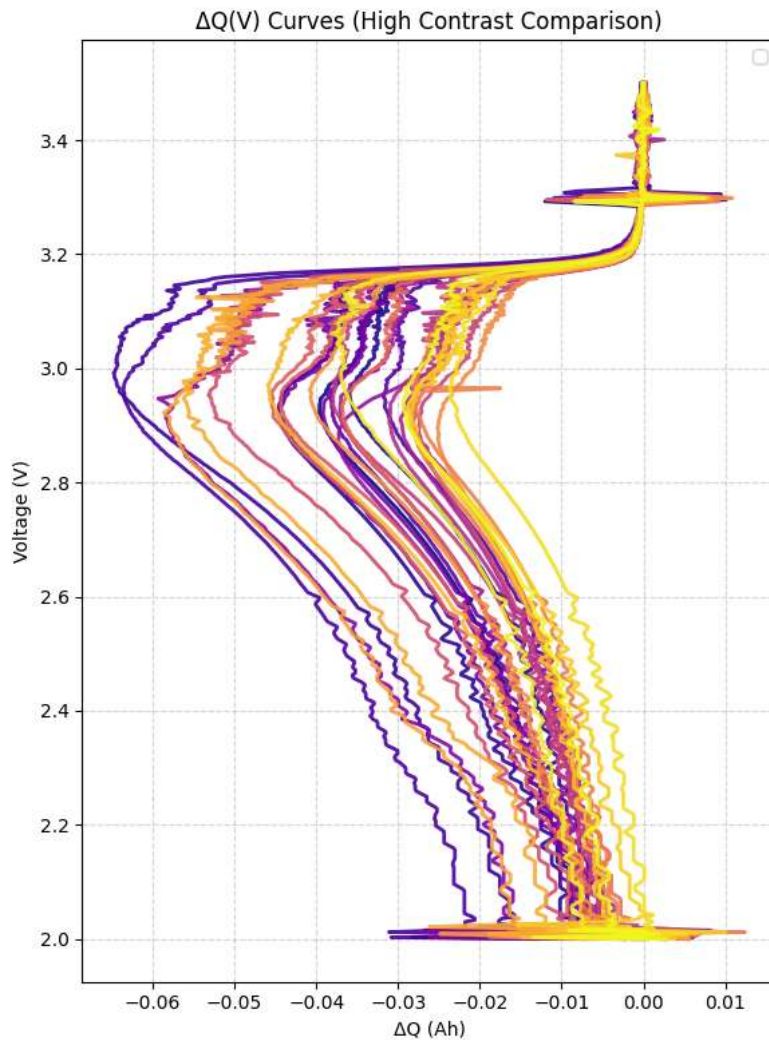
# styling
plt.xlabel("ΔQ (Ah)")
plt.ylabel("Voltage (V)")
plt.title("ΔQ(V) Curves (High Contrast Comparison)")

plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.tight_layout()

plt.show()

```

/tmp/ipykernel_3001/3530600883.py:58: UserWarning: No artists with labels found to put in legend. Note that artists whose label
plt.legend()



```
df[(df['Voltage'] > 2) & (df['Voltage'] < 3.5) & (df['Cycle_Index']== 100)].shape[0]
```

806

```
features = {}
```

```
features['deltaQ_mean'] = np.mean(delta_q)
features['deltaQ_var'] = np.var(delta_q)
features['deltaQ_min'] = np.min(delta_q)
features['deltaQ_max'] = np.max(delta_q)
```

```
print(features)
```

```
{'deltaQ_mean': np.float64(-0.01227218900984823), 'deltaQ_var': np.float64(9.449771645371294e-05), 'deltaQ_min': np.float64(-0.0
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d
```

```
voltage_grid = np.linspace(2.0, 3.5, 500)
```

```
plt.figure(figsize=(10,6))
```

```
colors = plt.cm.plasma(np.linspace(0, 1, len(df_list)))
```

```
delta_var_all = []
life_all = []
```

```

for i, df in enumerate(df_list):

    # cleaning
    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100)]
    df_ml = df[df['Cycle_Index'] > 0]

    if df_ml.empty:
        continue

    # cycles for ΔQ
    df1 = df_ml[(df_ml['Cycle_Index']==10) & (df_ml['Current'] < 0)]
    df2 = df_ml[(df_ml['Cycle_Index']==100) & (df_ml['Current'] < 0)]

    if df1.empty or df2.empty:
        continue

    # sort + unique
    df1 = df1.sort_values('Voltage').drop_duplicates('Voltage')
    df2 = df2.sort_values('Voltage').drop_duplicates('Voltage')

    try:
        # interpolation
        f1 = interp1d(df1['Voltage'], df1['Discharge_Capacity'],
                      fill_value="extrapolate")
        f2 = interp1d(df2['Voltage'], df2['Discharge_Capacity'],
                      fill_value="extrapolate")

        q1 = f1(voltage_grid)
        q2 = f2(voltage_grid)

        delta_q = q2 - q1

        # 🔥 variance feature
        delta_var = np.var(delta_q)

        # 📅 cycle life
        cycle_life = df_ml['Cycle_Index'].max()

        delta_var_all.append(delta_var)
        life_all.append(cycle_life)

        # plot
        plt.scatter(delta_var, cycle_life,
                    color=colors[i],
                    s=80,
                    alpha=0.9)

    except:
        continue

# convert to array
delta_var_all = np.array(delta_var_all)
life_all = np.array(life_all)

# 🔥 correlation (log life)
corr = np.corrcoef(delta_var_all, np.log(life_all))[0,1]

# 📈 best fit line (optional but recommended)
m, b = np.polyfit(delta_var_all, np.log(life_all), 1)
x_fit = np.linspace(delta_var_all.min(), delta_var_all.max(), 100)
y_fit = np.exp(m * x_fit + b)

plt.plot(x_fit, y_fit, color='black', linewidth=2, label='Fit')

# 📌 correlation text
plt.text(delta_var_all.min()*1.05,
         max(life_all)*0.8,
         f'Corr = {corr:.3f}',
         fontsize=12,
         bbox=dict(facecolor='white', alpha=0.7))

# styling (same clean look)
plt.yscale('log')

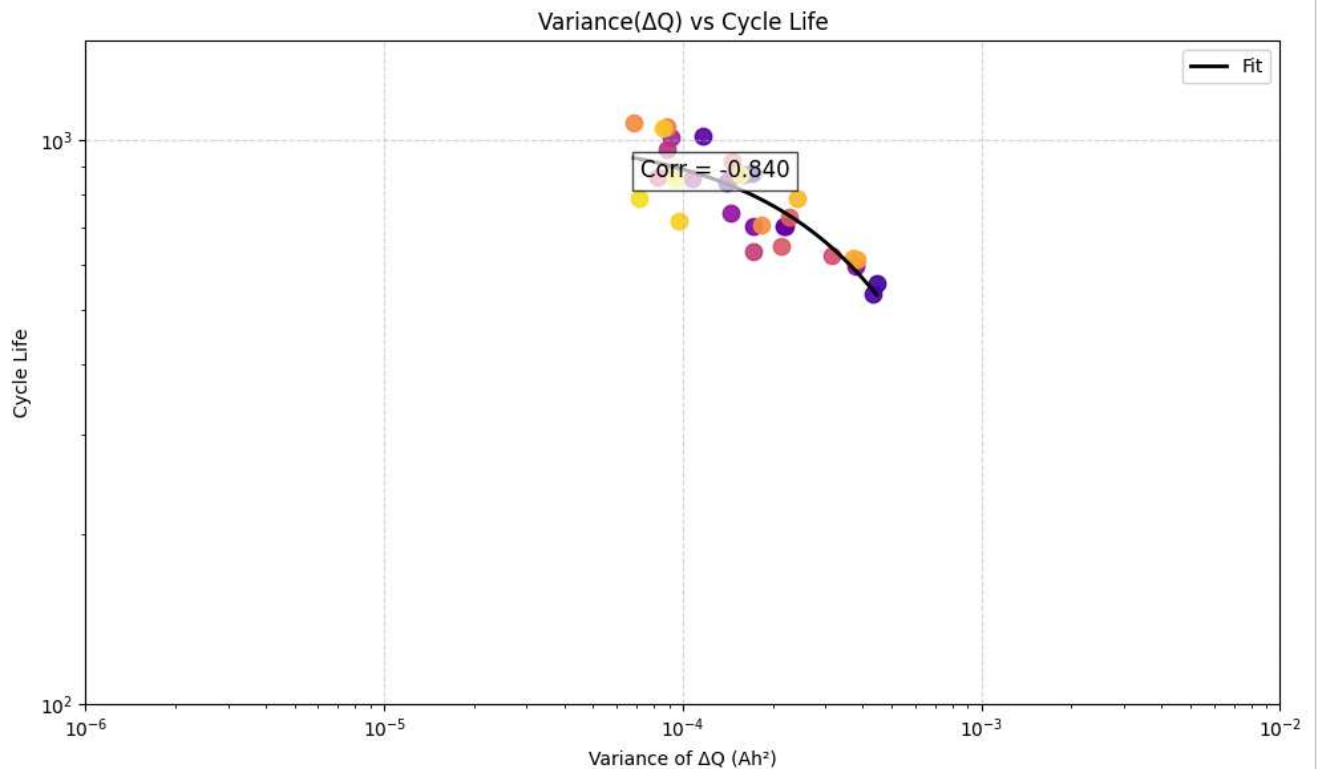
plt.xscale('log')
plt.xlim(10**-6, 10**-2)
plt.ylim(10**2, 1500)

```

```
plt.xlabel("Variance of  $\Delta Q$  (Ah2)")
plt.ylabel("Cycle Life")
plt.title("Variance( $\Delta Q$ ) vs Cycle Life")

plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.tight_layout()

plt.show()
```



```
#creating ml model 3 parameters are taken each battery
from scipy.interpolate import interp1d
from scipy.stats import linregress

voltage_grid = np.linspace(2.0, 3.5, 500)

feature_rows = []

for df in df_list:

    # cleaning
    df = df[(df['Voltage'] > 0) & (df['Temperature'] < 100)]
    df_ml = df[df['Cycle_Index'] > 0]

    if df_ml.empty:
        continue

    # 📄 cycle life (target)
    cycle_life = df_ml['Cycle_Index'].max()

    # -----
    # Feature 1: Capacity ratio
    # -----
    try:
        cap2 = df_ml[df_ml['Cycle_Index']==2]['Discharge_Capacity'].max()
        cap100 = df_ml[df_ml['Cycle_Index']==100]['Discharge_Capacity'].max()
        cap_ratio = cap100 / cap2
    except:
        continue

    # -----
    # Feature 2: Slope (95-100)
    # -----
    cycle_capacity = df_ml.groupby('Cycle_Index')['Discharge_Capacity'].max()
```

```

if not all(c in cycle_capacity.index for c in range(95,101)):
    continue

x = np.array(range(95,101))
y = cycle_capacity.loc[95:100].values
slope, _, _, _ = linregress(x,y)

# -----
# Feature 3: ΔQ variance
# -----
df1 = df_ml[(df_ml['Cycle_Index']==10) & (df_ml['Current']<0)]
df2 = df_ml[(df_ml['Cycle_Index']==100) & (df_ml['Current']<0)]

if df1.empty or df2.empty:
    continue

df1 = df1.sort_values('Voltage').drop_duplicates('Voltage')
df2 = df2.sort_values('Voltage').drop_duplicates('Voltage')

f1 = interp1d(df1['Voltage'], df1['Discharge_Capacity'], fill_value="extrapolate")
f2 = interp1d(df2['Voltage'], df2['Discharge_Capacity'], fill_value="extrapolate")

q1 = f1(voltage_grid)
q2 = f2(voltage_grid)

delta_q = q2 - q1
delta_var = np.var(delta_q)

# -----
# Add row
# -----
feature_rows.append({
    "cap_ratio": cap_ratio,
    "slope": slope,
    "delta_var": delta_var,
    "cycle_life": cycle_life
})

# final dataframe
df_features = pd.DataFrame(feature_rows)

print(df_features.head())
df_features

```


	cap_ratio	slope	delta_var	cycle_life
0	1.000649	-0.000039	0.000170	876.0
1	1.001832	0.000134	0.000141	842.0
2	1.002236	-0.000047	0.000217	704.0
3	0.994725	-0.000168	0.000446	559.0
4	0.989389	-0.000247	0.000434	534.0

	cap_ratio	slope	delta_var	cycle_life
0	1.000649	-0.000039	0.000170	876.0
1	1.001832	0.000134	0.000141	842.0
2	1.002236	-0.000047	0.000217	704.0
3	0.994725	-0.000168	0.000446	559.0
4	0.989389	-0.000247	0.000434	534.0
5	1.002881	-0.000018	0.000117	1017.0
6	1.001581	0.000012	0.000161	870.0
7	1.003157	-0.000038	0.000221	702.0
8	1.003033	-0.000029	0.000171	703.0
9	1.001727	-0.000046	0.000379	599.0
10	1.002602	-0.000049	0.000145	742.0
11	1.001807	-0.000043	0.000144	860.0
12	1.002221	-0.000022	0.000092	1014.0
13	1.004593	-0.000024	0.000107	857.0
14	1.002245	-0.000032	0.000089	966.0
15	1.003167	-0.000065	0.000172	636.0
16	1.004767	-0.000026	0.000083	862.0
17	1.002760	-0.000038	0.000316	625.0
18	1.001849	-0.000045	0.000214	648.0
19	1.001432	-0.000042	0.000226	731.0
20	1.002184	-0.000045	0.000146	917.0
21	1.004889	-0.000017	0.000091	880.0
22	1.004300	-0.000009	0.000088	1054.0
23	1.003394	-0.000022	0.000069	1074.0
24	1.001430	-0.000037	0.000183	709.0
25	0.999947	-0.000065	0.000382	616.0
26	1.001886	-0.000012	0.000372	617.0
27	1.001816	0.000146	0.000241	788.0
28	1.002972	-0.000028	0.000086	1051.0
29	1.006244	-0.000024	0.000097	719.0
30	1.005877	-0.000010	0.000072	788.0
31	1.001735	-0.000021	0.000155	870.0
32	1.002613	-0.000044	0.000094	854.0

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df_features[['cap_ratio', 'slope', 'delta_var']]
y = df_features['cycle_life']

# log transform (VERY IMPORTANT 🔥)
y_log = np.log(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_log, test_size=0.2, random_state=42
)

```

```

import matplotlib.pyplot as plt
import numpy as np

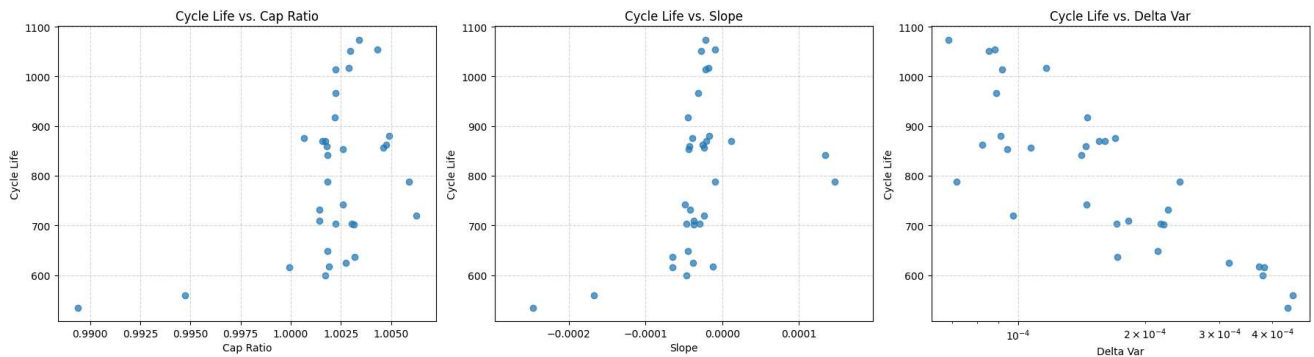
features = ['cap_ratio', 'slope', 'delta_var']
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i, feature in enumerate(features):
    axes[i].scatter(X[feature], y, alpha=0.7)
    axes[i].set_xlabel(feature.replace('_', ' ').title())
    axes[i].set_ylabel("Cycle Life")
    axes[i].set_title(f"Cycle Life vs. {feature.replace('_', ' ').title()}")
    axes[i].grid(True, linestyle='--', alpha=0.5)

    if feature == 'delta_var':
        axes[i].set_xscale('log')
    elif feature == 'cap_ratio':
        axes[i].set_xlim(X[feature].min() - 0.001, X[feature].max() + 0.001)
    elif feature == 'slope':
        axes[i].set_xlim(X[feature].min() - 0.00005, X[feature].max() + 0.00005)

plt.tight_layout()
plt.show()

```



Start coding or [generate](#) with AI.

```

#scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```

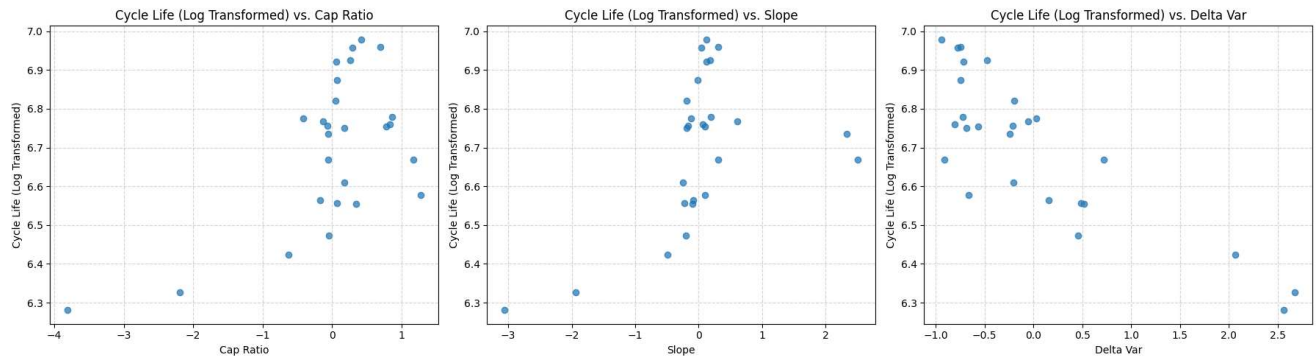
import matplotlib.pyplot as plt
import numpy as np

features = ['cap_ratio', 'slope', 'delta_var']
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i, feature in enumerate(features):
    axes[i].scatter(X_train[:, i], y, alpha=0.7)
    axes[i].set_xlabel(feature.replace('_', ' ').title())
    axes[i].set_ylabel("Cycle Life (Log Transformed)")
    axes[i].set_title(f"Cycle Life (Log Transformed) vs. {feature.replace('_', ' ').title()}")
    axes[i].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```



```
from sklearn.linear_model import ElasticNet

model_en = ElasticNet(alpha=0.01, l1_ratio=0.5)
model_en.fit(X_train, y_train)

pred_en = model_en.predict(X_test)
```

```
#elastic net regression
from sklearn.linear_model import ElasticNet

model_en = ElasticNet(alpha=0.01, l1_ratio=0.5)
model_en.fit(X_train, y_train)

pred_en = model_en.predict(X_test)
```

```
#random forest
from sklearn.ensemble import RandomForestRegressor

model_rf = RandomForestRegressor(n_estimators=200)
model_rf.fit(X_train, y_train)

pred_rf = model_rf.predict(X_test)
```

```
#gradient boost style
from sklearn.ensemble import GradientBoostingRegressor

model_gb = GradientBoostingRegressor()
model_gb.fit(X_train, y_train)

pred_gb = model_gb.predict(X_test)
```

```
#support vector regression
from sklearn.svm import SVR

model_svr = SVR(kernel='rbf')
model_svr.fit(X_train, y_train)

pred_svr = model_svr.predict(X_test)
```

```
#neural network (deep learning approach)
from sklearn.neural_network import MLPRegressor

model_nn = MLPRegressor(hidden_layer_sizes=(64,32), max_iter=500)
model_nn.fit(X_train, y_train)

pred_nn = model_nn.predict(X_test)
```

```

from sklearn.metrics import mean_squared_error, r2_score

def evaluate(y_test, pred, name):
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)

    print(f"{name}")
    print("RMSE:", rmse)
    print("R2:", r2)
    print("-----")

evaluate(y_test, pred_en, "ElasticNet")
evaluate(y_test, pred_rf, "RandomForest")
evaluate(y_test, pred_gb, "GradientBoosting")
evaluate(y_test, pred_svr, "SVR")
evaluate(y_test, pred_nn, "NeuralNet")

```

```

ElasticNet
RMSE: 0.09853366139220973
R2: 0.3471404766900813
-----
RandomForest
RMSE: 0.13444190724641183
R2: -0.21540167774254781
-----
GradientBoosting
RMSE: 0.1591711712672197
R2: -0.7036458740659073
-----
SVR
RMSE: 0.08391954261348311
R2: 0.5264381396352799
-----
NeuralNet
RMSE: 0.30525754294871316
R2: -5.265897844361221
-----

```

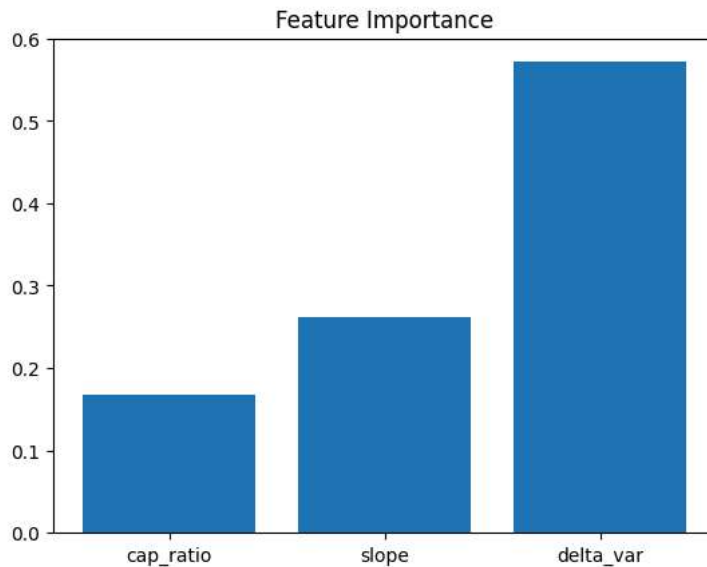
```

import matplotlib.pyplot as plt

importance = model_rf.feature_importances_
features = ['cap_ratio', 'slope', 'delta_var']

plt.bar(features, importance)
plt.title("Feature Importance")
plt.show()

```



```

from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR

models = {
    "Random Forest": RandomForestRegressor(n_estimators=100),

```

```
"Gradient Boosting": GradientBoostingRegressor(),
"Linear Regression": LinearRegression(),
"SVR": SVR()
```

```
model_results = {}

for name, model in models.items():

    # train
    model.fit(X_train, y_train)

    # predict
    pred = model.predict(X_test)

    # store results
    model_results[name] = {
        "model": model,
        "pred": pred
    }
```

```
plt.figure(figsize=(8,6))

colors = plt.cm.plasma(np.linspace(0,1,len(models)))

for i, (name, result) in enumerate(model_results.items()):

    pred = result["pred"]

    plt.scatter(
        np.exp(y_test),
        np.exp(pred),
        color=colors[i],
        label=name,
        alpha=0.7,
        s=60
    )

# ideal line
plt.plot([100,1500], [100,1500], 'k--')

plt.xlabel("Actual Life")
plt.ylabel("Predicted Life")
plt.title("Prediction vs Actual (All Models)")
plt.yticks(range(100,1501,100))
plt.xticks(range(100,1501,100))
plt.legend()
plt.grid()
plt.show()
```

